Assignment 1 — Search Functionality

Domains: E-commerce Website and Hospital Management System

I — Instruction

Generate at least 30 comprehensive test cases for search functionality across an e-commerce website and a hospital management application. Cover functional, negative, boundary, usability, security, compatibility, and performance testing.

C — Context

The e-commerce website allows users to search for products by name, brand, category, and keywords. Users can filter by price, brand, rating, and stock availability and sort by price, popularity, and newest arrivals.

The hospital application allows users to search for doctors, specialties, departments, and appointments. Filters include specialty, location, availability, and consultation type.

Test empty input, whitespace, special characters, misspellings, invalid inputs, no-result searches, duplicate results, large result sets, search accuracy, response times, and unauthorized access.

Assume the search response should complete within 2 seconds under normal test conditions. Treat this as a proposed performance requirement that must be confirmed.

E — Example

E-commerce examples:

Search for "wireless headphones" and verify relevant products.

Apply a brand filter and verify that all results match the selected brand.

Sort by price ascending and verify correct ordering.

Search for a nonexistent product and verify the no-results message.

Hospital examples:

Search for "cardiologist" and verify that matching specialties or doctors appear.

Filter doctors by availability and verify the returned appointment slots.

Search for an invalid doctor name and verify the appropriate empty state.

Attempt to access restricted patient information through search and verify that access is denied.

P — Persona

Act as a Senior QA Automation Engineer with expertise in web application testing, search functionality, negative testing, boundary-value analysis, security, performance, and automation.

O — Output Format

Generate at least 30 test cases across both applications. Use the following table columns:

Test Case ID

Application Domain

Test Scenario

Preconditions

Test Steps

Test Data

Expected Result

Actual Output

Status (Pass/Fail/Blocked/Not Executed)

Priority (High/Medium/Low)

Test Type

Automation Feasibility (Yes/No)

The Actual Output column must record the observed application behavior after execution. Do not populate it with the expected result or invent execution results. Leave it as "Not Executed" until the test is run.

Include positive, negative, boundary, security, usability, compatibility, and performance scenarios. Use measurable expected results and distinguish proposed requirements from confirmed requirements.

T — Tone

Use professional, precise, structured, and technically accurate language. Avoid vague test steps and expected results.

Assignment 2 — File Upload Functionality

Domain: Employee Management System

I — Instruction

Generate at least 25 comprehensive test cases for file uploads in an employee management system, covering successful uploads, validation, boundary conditions, error handling, security, usability, and performance.

C — Context

Authorized HR personnel can upload employee profile photos and supporting documents.

Supported formats are PDF, JPG, PNG, and DOCX. Assume the maximum file size is 10 MB per file unless the specification states otherwise.

Test valid files, unsupported formats, oversized files, zero-byte files, corrupted files, duplicate files, interrupted uploads, network failures, malicious files, double extensions, MIME-type mismatches, and unauthorized upload attempts.

Verify secure storage, correct employee association, and permission-based access to uploaded documents.

E — Example

Upload a valid 2 MB PDF and verify successful storage.

Upload a file at the maximum permitted size and verify the boundary.

Upload an 11 MB file and verify rejection with an appropriate message.

Upload an executable disguised with a .pdf extension and verify secure rejection.

Interrupt an upload and verify that incomplete files are not treated as successful uploads.

Attempt an upload without the required permission and verify access is denied.

P — Persona

Act as a Senior QA Automation Engineer specializing in file handling, UI/API testing, boundary-value analysis, security testing, and test automation.

O — Output Format

Generate at least 25 test cases in a table with:

Test Case ID

Test Scenario

Preconditions

Test Steps

Test Data

Expected Result

Actual Output

Status (Pass/Fail/Blocked/Not Executed)

Priority

Test Type

Automation Feasibility (Yes/No)

Actual Output must contain the observed result from execution, such as the displayed validation message, upload status, or resulting file state. Leave it as "Not Executed" before execution.

Include positive, negative, boundary, security, and performance cases. State any assumptions that require confirmation.

T — Tone

Use professional, precise, security-conscious language. Make all steps and expected results clear and measurable.

Assignment 3 — Shopping Cart Functionality

Domain: E-commerce Website

I — Instruction

Generate at least 30 comprehensive test cases for an e-commerce shopping cart, covering cart operations, price calculations, discounts, inventory, persistence, checkout readiness, and error handling.

C — Context

Customers can add products, remove items, update quantities, apply promotional coupons, and view subtotal, discounts, taxes, shipping charges, and the grand total.

The application must enforce product quantity limits and stock availability. All calculations must follow the configured business rules.

Test guest and authenticated users, invalid quantities, expired coupons, stock changes, session expiry, page refreshes, concurrent updates, and network interruptions.

E — Example

Add a product and verify its name, quantity, and price.

Update quantity from 1 to 3 and verify the recalculated subtotal.

Remove a product and verify that totals are updated.

Apply a valid coupon and verify the configured discount.

Apply an expired coupon and verify rejection with a meaningful message.

Attempt to add more units than available stock and verify the configured restriction.

Refresh the page and verify cart persistence according to the application rules.

P — Persona

Act as a Senior QA Engineer specializing in e-commerce testing, financial calculations, integration testing, and UI/API automation.

O — Output Format

Generate at least 30 test cases in a table with:

Test Case ID

Test Scenario

Preconditions

Test Steps

Test Data

Expected Result

Actual Output

Status (Pass/Fail/Blocked/Not Executed)

Priority

Test Type

Automation Feasibility (Yes/No)

Include calculation examples for subtotal, discount, tax, shipping, and grand total where applicable.

Actual Output must record observed cart values, messages, or behavior during execution. Leave it as "Not Executed" until execution.

Cover positive, negative, boundary, integration, concurrency, and recovery scenarios.

T — Tone

Use professional, structured, business-oriented language. Make calculation rules explicit and expected results verifiable.

Assignment 4 — Role-Based Access Control (RBAC)

Domain: Hospital Management System

I — Instruction

Generate at least 30 comprehensive test cases for role-based access control in a hospital management system. Verify that each user can perform only actions permitted by their assigned role.

C — Context

The application has the following roles:

Admin: Manage users, assign roles, and configure system settings.

Doctor: View assigned patient records and update permitted clinical information.

Receptionist: Register patients and manage appointments.

Viewer: Access explicitly permitted read-only information.

Create a permission matrix for patient records, appointments, clinical notes, user management, and reports.

Test UI and API authorization, direct URL access, unauthorized modifications, privilege escalation, session expiry, and role changes. Treat patient information as sensitive.

E — Example

Verify that an Admin can create users and assign approved roles.

Verify that a Doctor can update permitted clinical records.

Verify that a Receptionist cannot edit restricted clinical notes.

Verify that a Viewer cannot create, update, or delete records.

Access a restricted API endpoint directly and verify server-side authorization.

Change a user's role and verify that access reflects the updated permissions according to policy.

P — Persona

Act as a Senior QA Security Engineer specializing in RBAC, authentication, authorization, API security, and healthcare application testing.

O — Output Format

Generate at least 30 test cases in a table with:

Test Case ID

User Role

Feature or Resource

Required Permission

Preconditions

Test Steps

Expected Result

Actual Output

Status (Pass/Fail/Blocked/Not Executed)

Priority

Test Type

Automation Feasibility (Yes/No)

Include a role-permission matrix and separate UI-level and API-level authorization scenarios.

Actual Output must capture the observed response, such as an access-denied message, HTTP status, or resulting data state. Do not invent execution results.

T — Tone

Use formal, precise, security-focused language. Do not treat hidden UI controls as sufficient proof of authorization.

Assignment 5 — API Testing: User Registration

Domain: E-commerce REST API | Tools: Postman or Python with pytest and requests

I — Instruction

Generate at least 30 comprehensive API test cases for user registration. Cover HTTP methods, request validation, response validation, status codes, business rules, security, error handling, and negative testing.

C — Context

Assume the endpoint is:

```
POST /api/v1/users/register
```

Example request payload:

```
{ "name": "Test User", "email": "test@example.com", "password": "SecureExample123!", "phone":
"9876543210" }
```

Expected successful registration: HTTP 201 Created.

Validate required fields, data types, email format, password policy, phone number format, duplicate registration, malformed JSON, unsupported content types, and unexpected fields.

Explicitly test the following HTTP status codes:

200 OK: Successful request when the API contract specifies 200.

201 Created: Successful registration.

400 Bad Request: Malformed JSON, invalid payload structure, or invalid request data where specified by the contract.

401 Unauthorized: Missing or invalid authentication where authentication is required.

403 Forbidden: Authenticated caller lacks permission, where applicable.

404 Not Found: Invalid endpoint.

409 Conflict: Duplicate registration where defined by the API contract.

415 Unsupported Media Type: Unsupported content type.

422 Unprocessable Content: Semantically invalid input if supported by the API contract.

429 Too Many Requests: Rate-limit enforcement.

500 Internal Server Error: Unexpected server-side failure.

For HTTP 500 testing, use a controlled test environment or approved fault injection to simulate server failures. Verify that the API returns a safe error response without exposing stack traces, credentials, database details, or other sensitive information. Do not intentionally destabilize production.

A 500 response is not an acceptable result for an ordinary invalid user payload when the contract requires a client error such as 400 or 422.

E — Example

Submit a valid payload and verify HTTP 201 and the response schema.

Omit a required field and verify the documented client-error response.

Submit malformed JSON and verify HTTP 400 if specified by the contract.

Register an existing email and verify duplicate-account handling.

Call a protected endpoint without required authentication and verify HTTP 401.

Use an unsupported content type and verify HTTP 415.

Simulate a controlled server-side exception and verify safe HTTP 500 handling.

Verify that no password or password hash is exposed in the response.

Send repeated requests and verify configured rate limiting.

P — Persona

Act as a Senior API Test Automation Engineer experienced in REST APIs, HTTP standards, JSON Schema, Postman, Python, pytest, requests, and API security.

O — Output Format

Generate at least 30 test cases in a table with:

Test Case ID

Test Scenario

HTTP Method

Endpoint

Request Headers

Request Payload

Expected HTTP Status Code

Expected Response Body

Actual HTTP Status Code

Actual Response Body

Status (Pass/Fail/Blocked/Not Executed)

Priority

Test Type

Automation Feasibility (Yes/No)

Actual HTTP Status Code and Actual Response Body must capture the real response from execution. Before execution, mark them as "Not Executed." Never fabricate observed responses.

Include positive, negative, boundary, security, schema-validation, and server-error cases. Explain which API requirements must be confirmed before execution.

T — Tone

Use precise, structured, technically accurate language. Clearly distinguish expected behavior from observed behavior and confirmed requirements from assumptions.

Important API testing note: HTTP 400 and HTTP 500 represent different failure categories. A 400 generally indicates a client request problem; a 500 indicates an unexpected server-side failure. The exact status code for each scenario must follow the API contract.

Assignment 6 — Mobile OTP Login

Domain: Banking Mobile Application | Automation: Appium

I — Instruction

Generate at least 30 comprehensive test cases for OTP-based login in a banking mobile application. Cover authentication, OTP expiry, resend behavior, network resilience, device compatibility, security, and session management.

C — Context

The application supports Android and iOS. Users enter their registered phone number and receive an OTP for authentication.

Assume an OTP validity period of two minutes. The application has a resend cooldown and configurable attempt limits; exact values must be confirmed from the requirements.

Test valid, invalid, expired, reused, and incorrectly formatted OTPs. Include delayed messages, network failures, app backgrounding, restarts, Wi-Fi/mobile-data switching, and device compatibility.

Validate rate limiting, brute-force prevention, secure OTP handling, and session creation only after successful authentication.

E — Example

Enter a valid OTP before expiry and verify successful login.

Enter an incorrect OTP and verify authentication is denied.

Submit an OTP after two minutes and verify rejection.

Reuse a previously accepted OTP and verify rejection.

Request another OTP during the cooldown and verify configured behavior.

Disconnect the network during verification and verify graceful recovery.

Exceed the permitted verification attempts and verify the appropriate restriction.

P — Persona

Act as a Senior Mobile QA Automation Engineer specializing in banking applications, authentication security, Android, iOS, Appium, and network-condition testing.

O — Output Format

Generate at least 30 test cases in a table with:

Test Case ID

Test Scenario

Platform and OS Version

Preconditions

Test Steps

Test Data

Expected Result

Actual Output

Status (Pass/Fail/Blocked/Not Executed)

Priority

Test Type

Automation Feasibility (Yes/No)

Actual Output must capture observed app behavior, messages, authentication status, or recovery behavior. Leave it as "Not Executed" before testing.

Identify cases requiring real SMS delivery, device farms, or specialized test infrastructure.

T — Tone

Use precise, professional, security-conscious language. Do not expose real OTPs or sensitive banking information.

Assignment 7 — Dashboard and Report Generation

Domain: Hospital Management System

I — Instruction

Generate at least 30 comprehensive test cases for a hospital management dashboard and report-generation module. Cover data accuracy, filters, exports, permissions, formatting, performance, and error handling.

C — Context

The dashboard displays patient registrations, appointment counts, cancellations, and revenue metrics. Authorized users can filter reports by date range and export them in PDF, Excel, and CSV formats.

Validate date boundaries, invalid ranges, empty datasets, large datasets, time zones, and consistency between dashboard metrics and exported reports.

Test role-based visibility, restricted data fields, interrupted downloads, corrupted exports, and report-generation failures. Treat patient-related information as sensitive.

E — Example

Select a valid date range and verify dashboard metrics against a known test dataset.

Export a PDF and verify its content and readability.

Export Excel and CSV files and verify column values and record counts.

Select a date range without records and verify appropriate empty-state behavior.

Select an end date earlier than the start date and verify validation.

Compare dashboard totals with exported report totals.

Attempt a restricted export as an unauthorized user and verify access is denied.

P — Persona

Act as a Senior QA Engineer specializing in data validation, reporting systems, database integration, authorization, and UI/API automation.

O — Output Format

Generate at least 30 test cases in a table with:

Test Case ID

Report Type

Test Scenario

Preconditions

Test Steps

Test Data

Expected Result

Actual Output

Status (Pass/Fail/Blocked/Not Executed)

Priority

Test Type

Automation Feasibility (Yes/No)

Actual Output must record observed dashboard values, exported file details, displayed messages, or permission behavior. Leave it as "Not Executed" until execution.

Include data reconciliation examples and positive, negative, boundary, security, and performance cases. Mark any assumed requirements clearly.

T — Tone

Use professional, analytical, and technically precise language. All expected results must be measurable and actual results must be based on execution evidence.